```
$words = ['weak', 'strong', 'sauce', 'pickles', 'banana', 'infosec'];

foreach ($words as $word1) {
    testHash($word1, $hash);
    foreach ($words as $word2) {
        testHash($word1 . $word2, $hash);
    }
}

function testHash(string $word, string $hash): void {
    $ourHash = sha1($word);
    if ($ourHash == $hash) {
        exit("Password found! {$ourHash} / {$word}" . PHP_EOL);
    }
}

Output:
Password found! d31713bfca0fb0ca1d52ab365a5e70327d5a8942 / weaksauce
```

With a list of words and a trivial rule (the password is either one of the words or a concatenation of two words), we find the password. The code above is a very simplified form of *dictionary attack*, where common words are used to build test passwords, which are then passed through the hashing algorithm. Of course, we're encouraged to use substitute characters, punctuation, and so on when we create real passwords. This increase in complexity adds to the computational work an attacker needs to perform, but it's just a matter of time until the correct hash is found. To make matters worse, given what we've seen so far, we actually only need to generate a map of hash values to input strings once, after which we can simply reuse our pre-computed data. To add insult to injury, such pre-computed hash databases (known as *rainbow tables*) are available, saving an attacker the cost of initial computation. (Note: rainbow tables are a bit more sophisticated than simply a store of every potential password and hash value; they trade space for computation and are beyond the scope of this text. Just accept that their existence makes life difficult!)

So, what to do? Sure, we can add work to the hashing: the SHA1 is fast by modern standards, but there are other, more expensive algorithms available (the cost to us is irrelevant; we only need to make the calculation once per login), and we can add more